Egypt-Japan University of Science and Technology (E-JUST)

Department of Computer Science & Information Technology

AID 325 — Blockchain Applications

FreelanceChain

Decentralized Freelance Escrow Platform

Final Project Report

Team Member 1: Mohamed Yasser Ibrahim Saddah (ID: 320210004)

Team Member 2: Anas Bahloul Ahmed Salem (ID: 320210079)

Team Member 3: Mina Morcos Samir (ID: 320210043)

Submitted to:	Dr. Muhammad Hataba, Eng. Salma Walid
Submission Date:	May 2025
Network:	Ethereum Sepolia Testnet

All transactions verifiable at `sepolia.etherscan.io`

Contents

1 Team Configuration & Contract

1.1 Team Members

Member	Full Name	Student ID	Role
Member 1	Mohamed Yasser Ibrahim Saddah	320210004	Smart Contracts
Member 2	Anas Bahloul Ahmed Salem	320210079	Frontend
Member 3	Mina Morcos Samir	320210043	Backend & Testing

1.2 Team Goals

The team's primary goal was to design and deliver a functional decentralized freelance escrow platform that demonstrates genuine blockchain integration. Beyond meeting the course requirements, we aimed to build something that addresses a real problem — the lack of trust and transparency in existing freelance platforms. All three members committed to developing hands-on experience in Solidity smart contract development, full-stack Web3 application development, and hybrid blockchain-database architecture design.

1.3 Expectations & Policies

Communication: Daily communication through WhatsApp for quick updates, with weekly progress meetings. All project files shared through a common repository.

Deadlines: Internal deadlines set 2 days before official deadlines to allow time for review, testing, and fixes.

Meeting Attendance: All members must attend every scheduled meeting. Absences must be communicated at least 3 hours in advance. Two unexcused absences result in escalation to the instructor.

Work Quality: Each member is responsible for delivering their assigned tasks at an acceptable quality for review by the team before submission.

1.4 Addressing Non-Performance

If a member is underperforming, the team first holds an informal discussion to understand the issue. If the problem continues, a written notice is sent in the group chat documenting the concern. If the member still fails to meet responsibilities, they are required to

compensate the team for any costs or time lost — calculated and agreed upon by the remaining members. If unresolved, the matter is escalated to the instructor.

2 Updated Proposal & Research Background

2.1 Research Profiles

Mohamed Yasser is focused on Security Operations and incident response, with professional goals oriented toward SOC analysis and real-time threat monitoring. His graduation project on lightweight cryptographic algorithms on FPGAs gave him a strong foundation in hardware-level security, which motivated an interest in blockchain as a tamper-proof logging layer for cryptographic operations.

Anas Bahloul specializes in IT infrastructure and network administration. His graduation project, MalCatch, is a comprehensive malware analysis tool combining static and dynamic analysis. The idea of using blockchain to create an immutable audit trail for malware analysis reports — where each finding is recorded on-chain and cannot be altered — directly informed his approach to this project.

Mina Morcos is in the final stage of undergraduate studies with a focus on ethical hacking and vulnerability assessment. His goal is penetration testing, with particular interest in identifying security weaknesses in real-world systems. This project provided practical experience in smart contract security analysis and adversarial modeling.

2.2 The Blockchain Idea — Problem & Motivation

Centralized freelance platforms like Upwork and Fiverr act as intermediaries between clients and freelancers. While they provide a marketplace, they introduce several significant problems. Platform fees typically range from 10 to 20 percent of every transaction. Funds are held by the platform, meaning both parties must trust a company rather than a verifiable system. Dispute resolution is opaque and often biased toward the platform's commercial interests. Perhaps most critically, a freelancer's reputation is tied to the platform — it can be manipulated, removed, or simply lost if the platform shuts down.

The core question we asked was: *what if you could replace the company with code?* A smart contract that holds funds, enforces rules, and executes automatically removes the need for trust in any centralized party. The code is the law.

2.3 Scope Updates Since April 29 Submission

The project scope was enhanced in three areas compared to the original proposal:

1. **Timeout function added:** The original proposal did not include protection against client griefing (client refusing to approve or dispute). We implemented a `claimAfterTimeout()` function allowing freelancers to claim payment after 7 days of client inactivity.

2. **Gas optimization:** The Solidity optimizer was enabled with 200 runs in the Truffle configuration, reducing deployment and function call costs by approximately 15% compared to unoptimized builds.

3. **Expanded testing:** The original proposal mentioned unit testing without specifying scope. Final implementation includes 9 unit tests covering happy path, edge cases, and dispute resolution scenarios.

3 Project Description

3.1 Background

FreelanceChain is a decentralized escrow platform built on the Ethereum blockchain. It replaces the role of a centralized freelance platform with two Solidity smart contracts that execute automatically, transparently, and without any possibility of interference. When a client posts a job, ETH is locked inside the contract. It can only be released to the freelancer upon client approval, returned to the client through dispute resolution, or claimed by the freelancer after a timeout period. The platform owner collects a 2% fee only on successfully completed jobs.

3.2 The Real-World Problem

Three concrete problems motivated this work. First, the fee problem: Upwork charges between 10 and 20 percent. Our platform charges 2 percent, hardcoded into the contract and impossible to change after deployment. Second, the trust problem: on centralized platforms, both parties trust a company. If the company behaves badly, neither party has recourse. Our contract enforces the rules regardless of what anyone wants — including us. Third, the reputation problem: a freelancer's reputation on Upwork is owned by Upwork. Our platform stores reputation as ERC20 tokens in the freelancer's own wallet. Even if our platform shuts down, the tokens remain on Ethereum forever.

3.3 Expected Impact

The platform demonstrates that for specific use cases — holding and releasing funds based on verifiable conditions — the blockchain provides a technically superior solution to a centralized intermediary. It is not a replacement for everything a freelance platform does, but it replaces the most critical and most problematic part: the escrow.

4 Architectural Model

4.1 System Overview

The system consists of three layers working together. The first layer is the Ethereum blockchain, which stores all critical data and executes all payment logic. The second layer is a Node.js backend connected to MongoDB Atlas, which stores non-critical descriptive data. The third layer is a React.js frontend that connects the user's MetaMask wallet to both systems.

Layer	Technology	Responsibility
Blockchain	Solidity 0.8.20 + Ethereum Sepolia	ETH custody, job state, reputation tokens
Backend	Node.js + Express + MongoDB Atlas	Job titles, descriptions, profiles
Frontend	React.js + Web3.js + MetaMask	User interface, wallet signing

4.2 Smart Contract Layer

4.2.1 FreelanceEscrow.sol

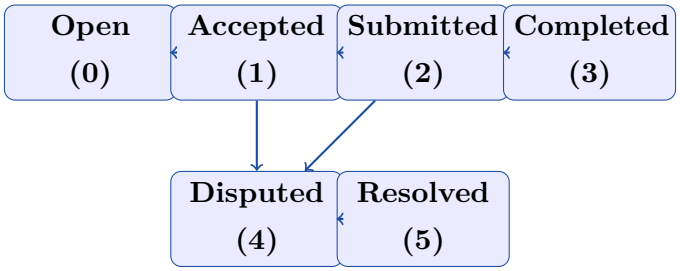
The main contract stores all jobs using a `mapping(uint256 => Job)` where the key is a numeric job ID generated by the frontend. The `Job` struct contains six fields:

```
struct Job {
    uint256   jobId;       // Links to MongoDB document
    address payable client;
    address payable freelancer;
    uint256   payment;     // Locked ETH in wei
    JobState  state;       // Current lifecycle state (0-5)
    uint256   createdAt;   // Block timestamp
}
```

A second mapping tracks completed jobs per address: `mapping(address => uint256) public completedJobs`, which provides an on-chain job completion counter independent of the REP token balance.

4.2.2 Job State Machine

Jobs transition through six states in a strictly controlled sequence. No state can be skipped and no backwards transition is possible.



4.2.3 RepToken.sol

The REP token is a standard ERC20 contract that inherits from OpenZeppelin's `ERC20` and `Ownable`. Its minting function is restricted exclusively to the `FreelanceEscrow` contract, which becomes the token's owner at deployment time. This means no external account can mint tokens — REP can only be awarded by the escrow logic itself.

```
function awardReputation(address freelancer, uint256 amount)
    external
    onlyOwner  // Only FreelanceEscrow can call this
{
    _mint(freelancer, amount);
    emit ReputationAwarded(freelancer, amount);
}
```

4.3 Deployed Contract Addresses (Sepolia Testnet)

Contract	Address
FreelanceEscrow	`0x74b13cE417d2b61a5809EACDBDE22cB552F84012`
RepToken	`0x378dE82774Bd313cE54DdfBD43Db6B379fd4369A`

4.4 Data Model

The system uses a hybrid data model. Critical data that requires trust — meaning money, ownership, and state — is stored on-chain. Non-critical descriptive data is stored in MongoDB.

On-Chain (Smart Contract)	Off-Chain (MongoDB Atlas)
Job ID	Job title
Client wallet address	Job description
Freelancer wallet address	Required skills list
Locked ETH amount (wei)	Client wallet (cached reference)
Current job state (0–5)	Freelancer wallet (cached)
Block timestamp	Transaction hash
Completed job count per address	User profiles
REP token balance per address	Creation timestamps

4.4.1 The Bridge — Shared Job ID

The `jobId` is the mechanism that links both systems. When a client creates a job through the frontend, the following sequence occurs:

1. Frontend generates `jobId = Date.now()` — a numeric Unix timestamp in milliseconds (example: `1778585969035`).
2. Frontend calls `contract.methods.createJob(jobId).send({value: paymentWei})` — this locks ETH and records the ID on-chain.
3. After the blockchain transaction confirms, frontend calls `POST /api/jobs` on the Node.js backend, sending the same `jobId` alongside the title, description, and skills.
4. MongoDB stores the descriptive data with the shared ID.

Now both systems are linked. The frontend can display complete job information by reading the title from MongoDB and the ETH state from the blockchain using the same ID.

4.5 Backend API Endpoints

Method	Endpoint	Description
GET	`/api/jobs`	Returns all jobs sorted by creation date
GET	`/api/jobs/:jobId`	Returns single job by numeric ID
POST	`/api/jobs`	Saves new job after blockchain transaction
PUT	`/api/jobs/:id/accept`	Updates freelancer wallet after on-chain accept
GET	`/health`	Server health check

4.6 Frontend Architecture

The frontend is a React single-page application using `react-router-dom` for navigation. Wallet state is managed through a React Context (`Web3Context`) that provides the Web3 instance, connected account, and contract object to all components.

Four main pages cover the full user journey:

- **Browse Jobs** — reads job list from MongoDB, reads on-chain state for each job, displays combined view
- **Post a Job** — form that triggers `createJob()` transaction and saves to MongoDB on confirmation
- **Job Detail** — shows full job data with context-aware action buttons based on caller's role and current state
- **My Profile** — reads REP token balance and job history directly from blockchain

5 Security Analysis

5.1 Reentrancy Attack Prevention

The most significant vulnerability in escrow contracts is reentrancy. When a contract sends ETH to an external address, the receiving address could be a malicious contract that immediately calls back into the sender before the sender's state is updated. In an escrow context, this means draining all locked funds in a single transaction.

We applied two independent layers of protection.

Layer 1 — OpenZeppelin ReentrancyGuard: Both `approveWork()` and `resolveDispute()` use the `nonReentrant` modifier. This modifier sets an internal flag before execution and reverts any call that attempts to re-enter while the flag is active.

Layer 2 — Checks-Effects-Interactions Pattern: All payment functions follow a strict ordering. State variables are updated *before* any external call is made.

```
function approveWork(uint256 jobId) external onlyClient(jobId)
    inState(jobId, JobState.Submitted) nonReentrant
{
    // CHECKS
    uint256 total = job.payment;
    require(total > 0, "Escrow: no payment to release");

    // EFFECTS (state updated BEFORE any transfer)
    job.state   = JobState.Completed;
    job.payment = 0;
    completedJobs[job.freelancer]++;

    // INTERACTIONS (external calls happen last)
    job.freelancer.transfer(payout);
    payable(owner).transfer(fee);
    repToken.awardReputation(job.freelancer, REP_PER_JOB);
}
```

Even if reentrancy were to occur, `job.payment` is already set to zero before any transfer, so a re-entrant call would find no funds to withdraw.

5.2 Access Control — Custom Modifiers

Five custom modifiers enforce role-based access control throughout the contract.

Modifier	What It Enforces
`onlyOwner`	Restricts to the platform owner (deployer)
`onlyValidator`	Restricts to the designated dispute validator
`onlyClient(jobId)`	Restricts to the client of a specific job
`onlyFreelancer(jobId)`	Restricts to the assigned freelancer of a specific job
`inState(jobId, state)`	Requires the job to be in a specific state

The combination of `onlyClient` and `inState(Submitted)` on `approveWork()` means that only the client of that specific job can call the function, and only when the job is in the

Submitted state. If either condition fails, the transaction reverts.

5.3 Input Validation

Every public function uses `require()` statements with descriptive error messages. This ensures that invalid inputs are caught early, gas is not wasted, and the caller receives meaningful feedback.

```
require(msg.value > 0, "Escrow: payment must be greater than zero");
require(jobs[jobId].client == address(0), "Escrow: job ID already
    exists");
require(msg.sender != jobs[jobId].client,
        "Escrow: client cannot be their own freelancer");
require(_validator != address(0),
        "Escrow: validator cannot be zero address");
```

5.4 Immutable and Constant Variables

Sensitive addresses and fixed parameters are declared with `immutable` or `constant`, preventing any modification after deployment and reducing gas costs on reads.

```
address public immutable owner;      // Set in constructor, never
    changes
address public immutable validator;  // Set in constructor, never
    changes
RepToken public immutable repToken;  // Set in constructor, never
    changes
uint256 public constant FEE_PERCENT = 2;
uint256 public constant REP_PER_JOB = 1 * 10**18;
uint256 public constant TIMEOUT_DAYS = 7;
```

5.5 Timeout Protection Against Client Griefing

A vulnerability identified during development is the griefing scenario: a client accepts a freelancer's work but never calls `approveWork()` and never raises a dispute, leaving the freelancer's payment locked indefinitely. We addressed this directly by implementing `claimAfterTimeout()`:

```
function claimAfterTimeout(uint256 jobId)
    external
    onlyFreelancer(jobId)
    inState(jobId, JobState.Submitted)
    nonReentrant
{
```

```
    require(
        block.timestamp >= job.createdAt + (TIMEOUT_DAYS * 1 days),
        "Escrow: timeout period has not passed yet"
    );
    // ... releases full payment to freelancer
}
```

If the client does not respond within 7 days of work submission, the freelancer can claim the full payment automatically. No validator or third party is required.

5.6 Threat Model

Threat	Risk	Mitigation
Reentrancy on `approveWork()`	High	`nonReentrant` modifier + CEI pattern. State set to zero before transfer.
Validator corruption	Medium	Validator can only route funds to client or freelancer — never to themselves. Future: multi-sig.
Client griefing	Medium	`claimAfterTimeout()` implemented. Freelancer can claim after 7 days.
Flash loan governance attack	Low	REP tokens have no voting power. Cannot be used to influence contract decisions.
Front-running `acceptJob()`	Low	Jobs are public by design. Acceptable risk for this use case.

5.7 Events for Off-Chain Auditability

Every state-changing transaction emits an event. These events are written permanently to the Ethereum blockchain and cannot be altered or deleted. They serve as an immutable audit trail.

```
event JobCreated(uint256 indexed jobId, address indexed client,
                 uint256 payment);
event JobAccepted(uint256 indexed jobId, address indexed freelancer);
event WorkSubmitted(uint256 indexed jobId, address indexed
    freelancer);
event PaymentReleased(uint256 indexed jobId, address indexed
    freelancer,
                      uint256 amount);
event DisputeRaised(uint256 indexed jobId, address indexed raisedBy);
```

```
event DisputeResolved(uint256 indexed jobId, address indexed winner,
                      uint256 amount);
event TimeoutClaim(uint256 indexed jobId, address indexed freelancer,
                   uint256 amount);
```

6 Implementation & Performance

6.1 Development Environment

Component	Version / Detail
Solidity	0.8.20
OpenZeppelin Contracts	v5.x
Truffle	v5.11.5
Ganache CLI	v7.9.2
Node.js	v20.20.2 (via NVM)
React.js	v18.2
Web3.js	v1.10.4 (pinned for stability)
MongoDB	Atlas M0 Free Cluster
Local OS	Ubuntu 24.x
Testnet	Ethereum Sepolia

6.2 Unit Test Results

Nine unit tests were written covering three scenario categories. All tests run on a local Ganache instance for speed and reproducibility.

#	Test Description	Result
1	Client can create a job and lock ETH	**PASS**
2	Freelancer can accept an open job	**PASS**
3	Full flow: create → accept → submit → approve releases ETH + REP	**PASS**
4	Cannot accept a job already in Accepted state	**PASS**
5	Client cannot approve work not yet submitted	**PASS**
6	Client cannot be their own freelancer	**PASS**
7	Cannot create a job with zero payment	**PASS**
8	Validator resolves dispute in favor of freelancer	**PASS**
9	Stranger cannot resolve a dispute	**PASS**
	Total	**9 / 9**

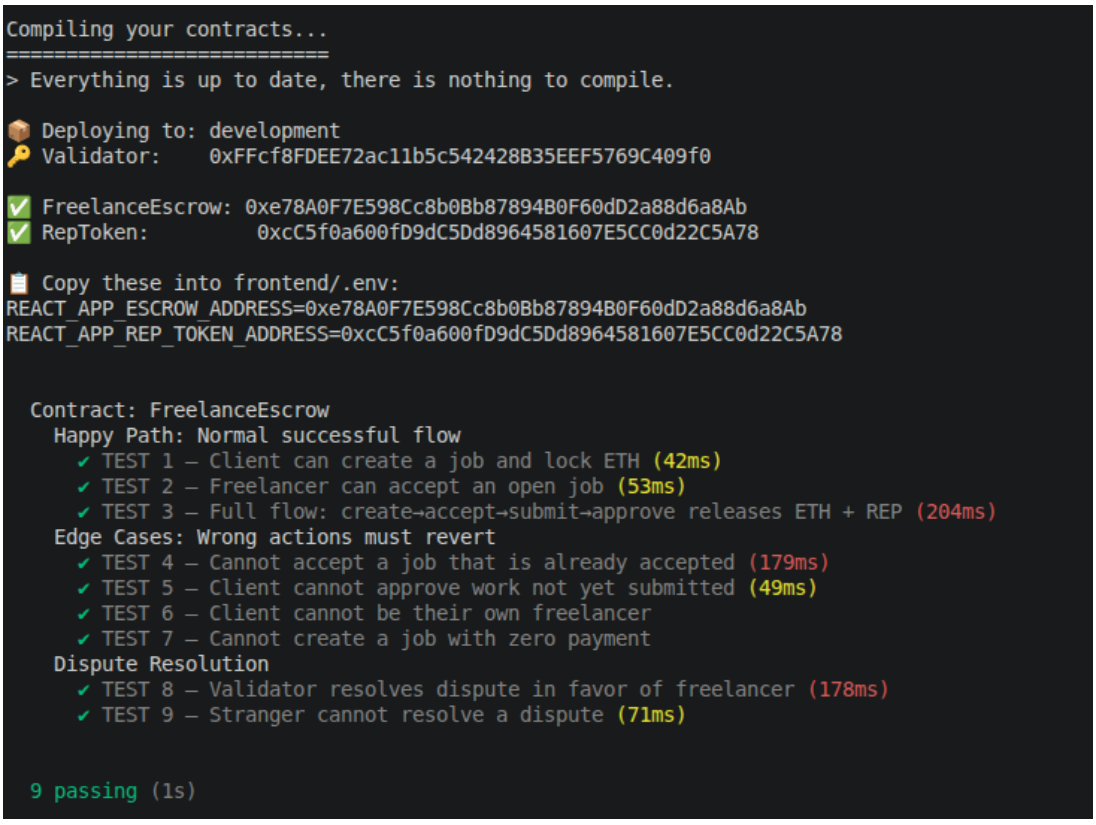


Figure 1: All 9 unit tests passing on local Ganache — `truffle test`

6.3 Deployment

6.3.1 Local Deployment (Ganache)

Before deploying to Sepolia, the contracts were tested on a local Ganache blockchain. This allowed rapid iteration without spending testnet ETH or waiting for block confirmations.

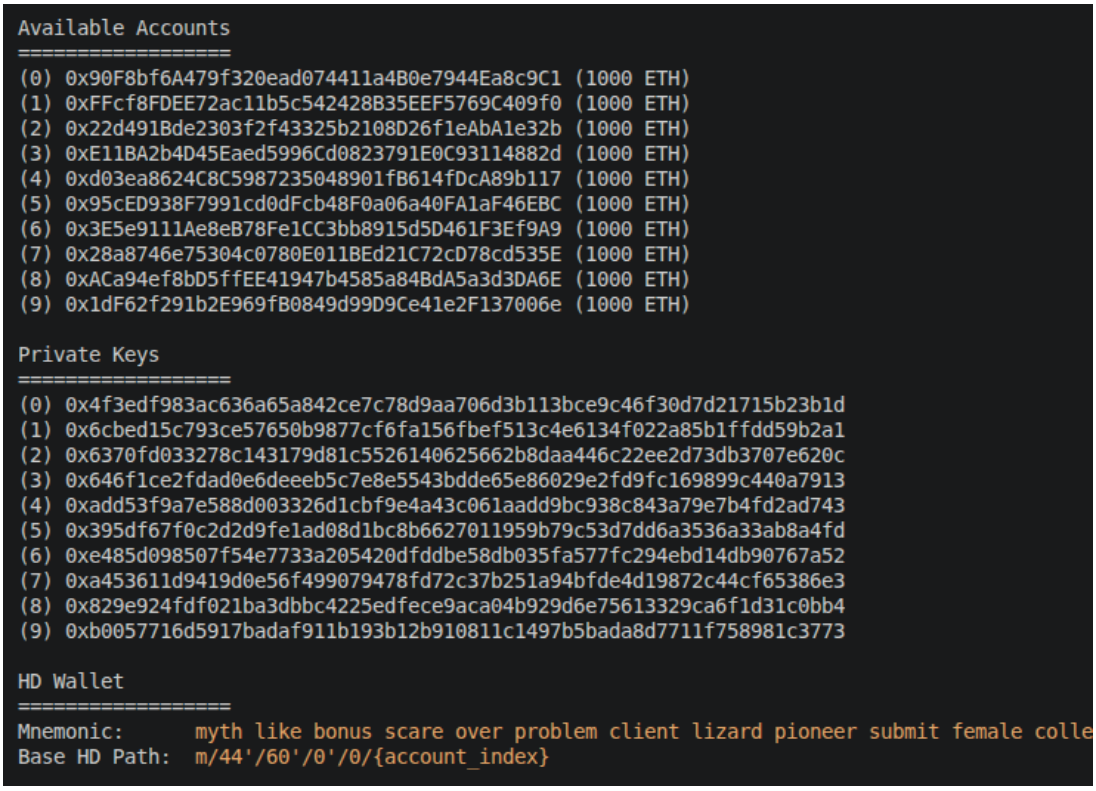


Figure 2: Ganache CLI running locally with 10 test accounts each holding 1000 ETH

6.3.2 Sepolia Testnet Deployment

Field	Value
Network	Ethereum Sepolia Testnet
FreelanceEscrow address	`0x74b13cE417d2b61a5809EACDBDE22cB552F84012`
RepToken address	`0x378dE82774Bd313cE54DdfBD43Db6B379fd4369A`
Deployment TX hash	`0x94ea689aee271a990a9c071b8749450ccb...`
Block number	10,834,738
Gas used (deployment)	3,235,541
Total cost	0.00808 ETH

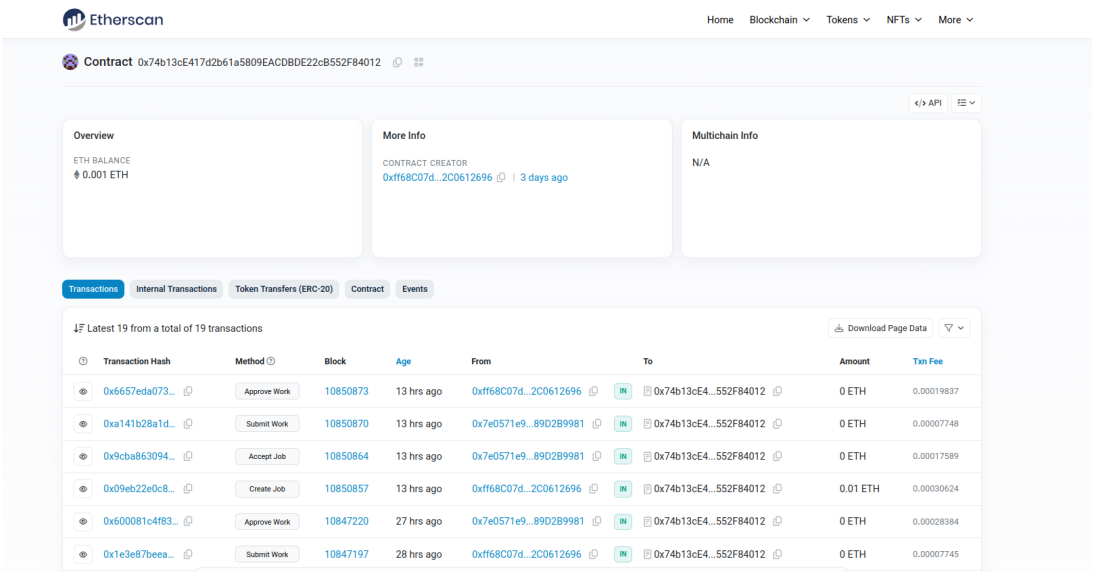


Figure 3: FreelanceEscrow contract deployed and visible on Sepolia Etherscan at `0x74b13cE417d2b61a5809EACDBDE22cB552F84012`

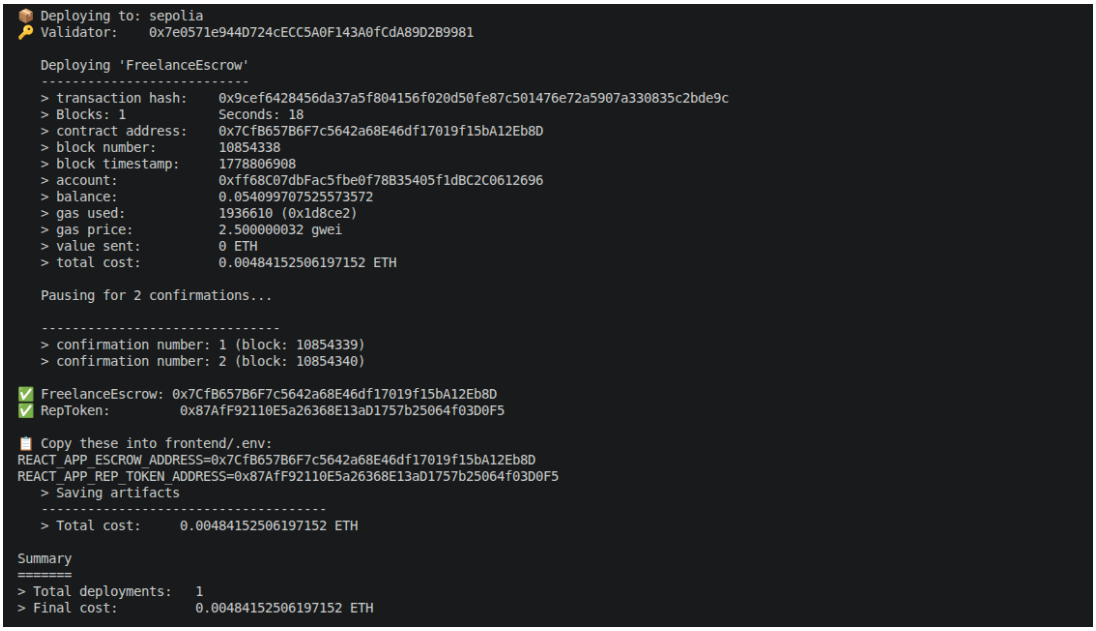


Figure 4: Successful Sepolia testnet deployment via `truffle migrate --network sepolia`

6.4 Gas Analysis

Gas costs were measured by deploying to a local Ganache instance and executing each function in sequence. The Solidity optimizer was enabled with 200 runs, which reduces gas usage for frequently called functions at the cost of slightly larger deployment bytecode.

Function	Gas Used	Cost at 2.5 Gwei	Why This Cost
`createJob()`	139,548	≈ 0.000349 ETH	Writes new struct to storage (SSTORE is most expensive EVM op)
`acceptJob()`	70,311	≈ 0.000176 ETH	Updates existing storage slot
`submitWork()`	30,947	≈ 0.000077 ETH	Single state variable change
`approveWork()`	130,601	≈ 0.000327 ETH	ETH transfer + external contract call (RepToken mint)
`resolveDispute()`	≈ 70,000	≈ 0.000175 ETH	Similar to acceptJob + transfer

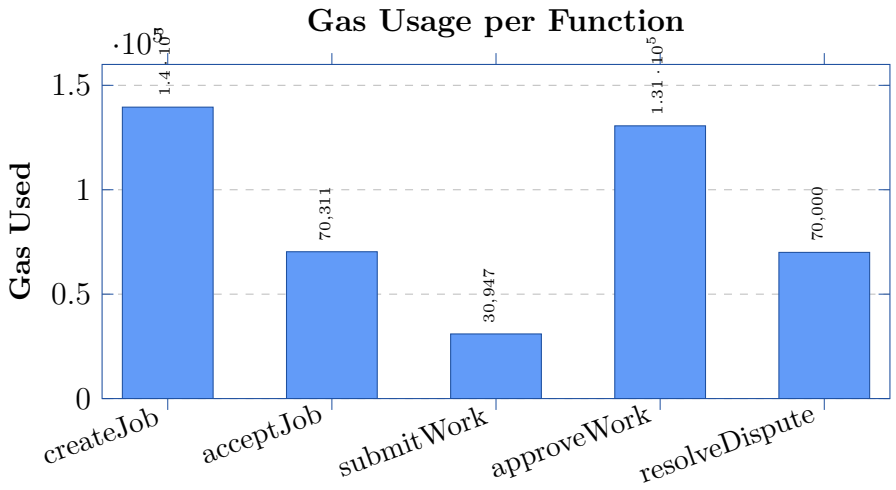


Figure 5: Gas consumption per contract function. `submitWork()` is cheapest because it only changes one state variable. `createJob()` is most expensive because it writes a new struct to storage for the first time.

Key observation: The pattern makes intuitive sense. Writing new data to blockchain storage (`createJob`) costs the most because the EVM's `SSTORE` opcode for new storage slots is the single most expensive operation in the EVM. Updating existing slots (`acceptJob`, `submitWork`) is cheaper because the slot already exists. `approveWork` is expensive despite not creating new storage because it performs an external call to `RepToken.awardReputation()`, which itself executes a `_mint()` operation.

6.5 Frontend Screenshots

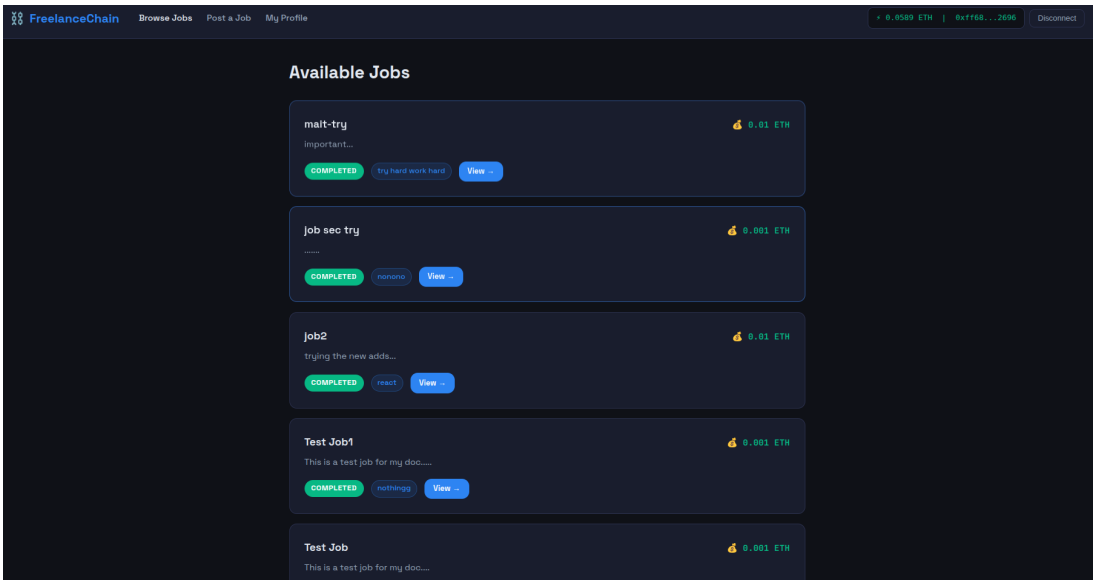


Figure 6: FreelanceChain Browse Jobs page showing live jobs with on-chain state badges

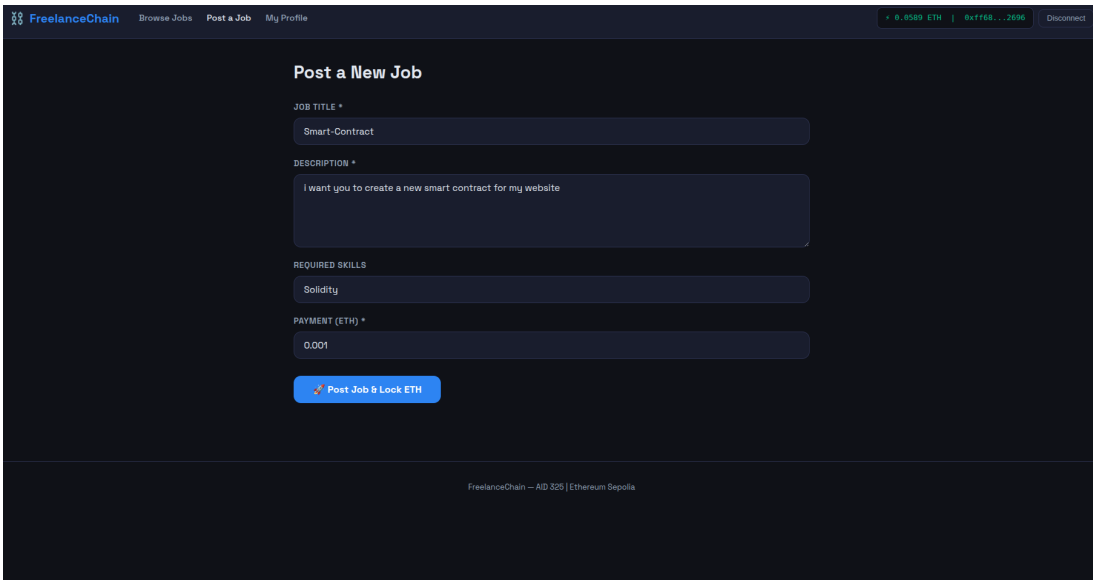


Figure 7: Post a Job form — client fills in job details before locking ETH in escrow

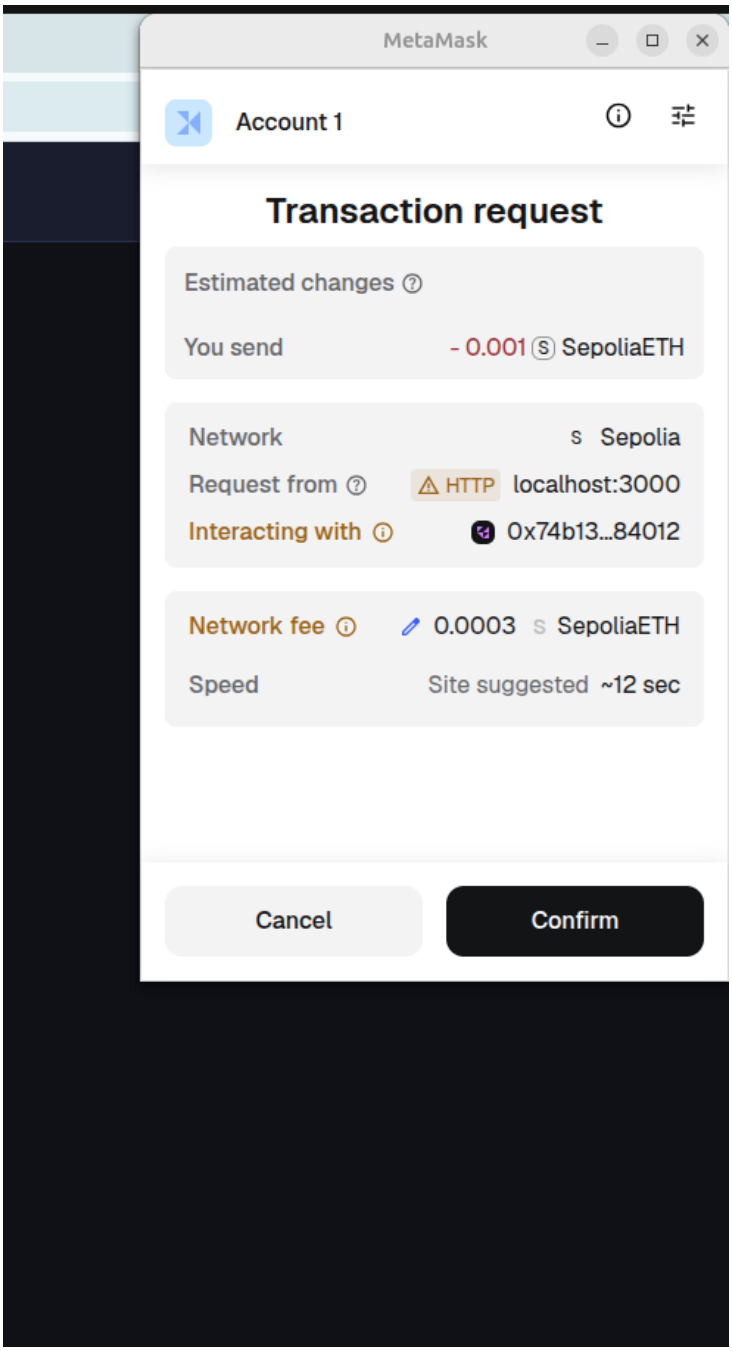


Figure 8: MetaMask transaction confirmation — user approves locking 0.001 ETH in the escrow contract

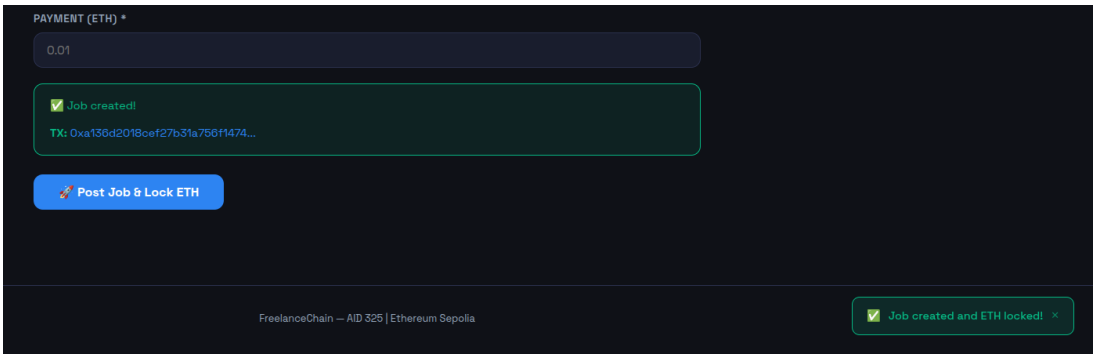


Figure 9: Transaction confirmed — TX hash displayed with Etherscan link for verification

Figure 10: My Profile page — REP token balance updated after job completion, read directly from the blockchain

6.6 NatSpec Documentation

All public and external functions include NatSpec documentation using the `@notice`, `@dev`, `@param`, and `@return` tags. This satisfies the formal documentation bonus requirement. Example from `approveWork()`:

```
/**
 * @notice Client approves submitted work, releasing ETH to freelancer
 * @param jobId The ID of the job to approve
 * @dev Uses nonReentrant + Checks-Effects-Interactions to prevent
     reentrancy.
 *      State is updated before any external transfer call.
 */
function approveWork(uint256 jobId)
    external
    onlyClient(jobId)
    inState(jobId, JobState.Submitted)
    nonReentrant
{
    // ...
}
```

7 Video Demonstration

Field	Value
YouTube URL	`[INSERT YOUTUBE LINK HERE]`
Duration	`[INSERT DURATION, e.g. 6 minutes 45 seconds]`
Visibility	Unlisted

The video covers the following sections:

1. Project introduction and problem statement (0:00 – 0:45)
2. Smart contract code walkthrough — state machine, modifiers, ReentrancyGuard (0:45 – 2:00)
3. Live demo: posting a job with MetaMask, loading state, TX hash (2:00 – 3:30)
4. Live demo: accepting, submitting, approving — full payment flow (3:30 – 5:00)
5. Etherscan verification of all transactions (5:00 – 6:00)
6. Summary (6:00 – end)

References

1. Ethereum Foundation. *Solidity Documentation, v0.8.20*. 2024. https://docs.soliditylang.org
2. OpenZeppelin. *Contracts v5.0 Documentation*. 2024. https://docs.openzeppelin.com/contracts/5.x
3. Consensys. *MetaMask Developer Documentation*. 2024. https://docs.metamask.io
4. Nakamoto, S. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.
5. Buterin, V. *Ethereum Whitepaper*. 2014. https://ethereum.org/en/whitepaper
6. Trufflesuite. *Truffle Documentation*. 2024. https://trufflesuite.com/docs
7. Wood, G. *Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)*. 2014.
8. OpenZeppelin. *Reentrancy Guard*. https://docs.openzeppelin.com/contracts/5.x/api/utils#ReentrancyGuard

A Smart Contract Addresses Summary

Item	Value
FreelanceEscrow (Sepolia)	`0x74b13cE417d2b61a5809EACDBDE22cB552F84012`
RepToken (Sepolia)	`0x378dE82774Bd313cE54DdfBD43Db6B379fd4369A`
Deployment TX	`0x94ea689aee271a990a9c071b8749450ccb777ca6c...`
Network	Ethereum Sepolia (Chain ID: 11155111)
Etherscan	https://sepolia.etherscan.io/address/0x74b13cE417d2b61a5809EACDBDE22cB552F84012